

System Design Document: TinyPal App

1. Introduction

This document outlines the system design and architecture of the **TinyPal** mobile application. The application is designed to provide users with a dynamic and engaging story-like experience for consuming informational content, categorized into "Did You Know?" facts and "Flash Cards."

The core design philosophy is built on **modularity** and **reusability**. The architecture breaks down the user interface into a set of independent, reusable components that are composed together to build complex screens. This approach ensures that the application is scalable, maintainable, and easy to debug.

***Development Note:** AI-powered tools like Gemini and ChatGPT were utilized to assist in generating documentation and refining styling features throughout the project.*

2. High-Level Architecture

The application is built using **React Native** and follows a standard component-based architecture.

- **Navigation:** Managed by **React Navigation (@react-navigation/native-stack)**. The root App.js component defines a stack navigator that manages the flow between the HomeScreen and the content screens.
- **State Management:** Primarily handled within container components (screens) using React hooks (useState, useEffect, useRef). State and control functions are passed down to child components via props.
- **Data Fetching:** The axios library is used to perform asynchronous API calls to a Heroku backend to fetch card content dynamically.
- **Component Structure:** The project is organized into logical directories to separate concerns.

2.1. Project Folder Structure

The project's files are organized to clearly distinguish between screens (top-level views) and reusable components. This structure promotes a clean and scalable codebase.

```
TinyPals/
├── .expo/
├── assets/
│   ├── images/
│   └── data/
├── Components/
│   ├── Card/
│   │   └── Card.js
│   ├── Content/
│   │   └── Content.js
│   ├── FlashCardContent/
│   │   └── FlashCardContent.js
│   ├── Header/
│   │   └── Header.js
│   ├── ProgressBar/
│   │   └── ProgressBar.js
│   ├── TinuCard/
│   │   └── TinuCard.js
│   ├── TinuChip/
│   │   └── TinuChip.js
│   ├── TinuOverlay/
│   │   └── TinuOverlay.js
│   └── TinuSheet/
│       └── TinuSheet.js
├── node_modules/
├── Screens/
│   ├── DYK/
│   │   └── DYK.js
│   └── HomeScreen/
│       └── HomeScreen.js
└── App.js
```

3. Component Breakdown & Reusability Strategy

The design heavily emphasizes creating small, single-responsibility components that can be reused and composed in various ways.

3.1. Presentational Components (/Components)

These components are the reusable building blocks of the application.

- **Header.js:** A generic header component that displays a title and subtitle. It is used within Card.js to provide a consistent look and feel.
- **ProgressBar.js:** A specialized component that renders the story-style progress bars at the top of the screen. It is completely controlled by props from its parent.
- **Content.js & FlashCardContent.js:** These components are responsible for rendering the specific body content for "Did You Know?" and "Flash Card" types, respectively. This separation allows the main Card.js component to remain generic.
- **Tinu Feature Components (TinuSheet, TinuOverlay, TinuCard, TinuChip):** This suite of components builds the entire "Tinu" interactive overlay feature. They are designed to be self-contained; TinuSheet acts as the entry point, which then presents the TinuOverlay containing TinuCard and TinuChip components.

3.2. Container Components (/Screens)

These components assemble the presentational components and provide them with data and logic.

- **HomeScreen.js:** A simple screen that serves as the main navigation hub. Its only role is to present navigation options to the user.
- **Card.js (Component acting as a Container):** This component is a prime example of composition. It acts as a container for the main content area, composing the Header, TinuSheet, and conditionally rendering either the Content or FlashCardContent component based on the type prop.
- **DYK.js (Screen):** This is the most significant container component and the best example of reusability at the screen level.
 - **Logic Reusability:** It contains all the core logic for fetching data, pre-loading images, managing the active card index, and handling the 15-second progress timer.
 - **Feature Switching:** This single screen powers **both** the "Did You Know?" and "Flash Card" features. The type parameter, passed via navigation from App.js, tells the screen which data to fetch from the API and which content component to ultimately render via the Card.js component. This avoids code duplication and centralizes the story-viewing logic in one place.

4. Key Feature Implementation

4.1. 15-Second Card Progression Timer

The automatic progression of cards is a core feature, designed to make each card visible for exactly 15 seconds. This is implemented within the useEffect hook in DYK.js.

- **Mechanism:** A setInterval is initialized to run every 750ms.
- **Progress Calculation:** On each tick of the interval, a progress counter is incremented by 5 (out of 100).
- **Duration:** To reach 100% progress, it takes 20 ticks ($100 / 5 = 20$). The total duration is therefore $20 \text{ ticks} * 750\text{ms/tick} = 15000\text{ms}$, which is equal to **15 seconds**.
- **State Update:** The progress state is updated on each tick, which is passed as a prop to

the `ProgressBar.js` component to visually render the filling of the active card's progress bar. When progress exceeds 100, the logic advances to the next card.

4.2. 1:3:1 Interactive Tap Zones

To provide intuitive user controls for navigating and pausing the stories, the screen in `Card.js` is divided into three invisible, pressable zones layered on top of the card content.

- **Layout Ratio:** The screen width is programmatically divided into a **1:3:1 ratio**:
 - **Left Zone:** Occupies the first 20% of the screen width.
 - **Middle Zone:** Occupies the central 60% of the screen width.
 - **Right Zone:** Occupies the final 20% of the screen width.
- **Implementation:** This is achieved using three `<Pressable>` components with absolute positioning:
 1. **Previous Card:** The left `Pressable` component covers width: '20%'. On press, it calls the `handleLeft` function passed down from `DYK.js`.
 2. **Pause/Resume:** The middle `Pressable` component covers width: '60%'. It uses the `onPressIn` and `onPressOut` events to call `setPaused(true)` (on press down) and `setPaused(false)` (on release), respectively. This allows the user to temporarily pause the 15-second timer by holding their finger on the screen.
 3. **Next Card:** The right `Pressable` component covers width: '20%'. On press, it calls the `handleRight` function.

This design provides a large, easily accessible area for the most common action (pausing) while still offering intuitive controls for navigation, mimicking the user experience of popular story-based social media apps.